

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week2 - Monday	Time(s)	
Duration	2 Hours	Applicable to Batches	24CSBTB01 To 24CSBTB39
Assignment Number:3.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab Experiment: Prompt Engineering – Improving Prompts and Context Management (0.5 marks) Objective	Week2 - Monday	

	To explore how prompt design and context influence AI-generated outputs and to learn techniques to improve AI responses. Tools Required ➔ GitHub Copilot / Google Gemini / ChatGPT ➔ VS Code / Google Colab ➔ Internet access Procedure ➔ Select a simple task: <i>"Write a Python function to check if a number is prime."</i> ➔ Use different prompting strategies to generate the solution: ➔ Zero-Shot – no examples. ➔ One-Shot – one example provided. ➔ Few-Shot – multiple examples provided. ➔ Context-Managed – detailed prompt with constraints and instructions. ➔ Record AI responses and refine prompts to improve code quality. ➔ Request AI to optimize the logic for efficiency. ➔ Compare results and document improvements. Sample Prompts ➔ Zero-Shot: Write a Python function to check if a number is prime. ➔ <i>One-Shot:</i> <i>Example: Input: 5 → Output: Prime. Now, write a function to check if a number is prime.</i> ➔ Few-Shot:	
--	---	--

Example 1: Input: 7 → Output: Prime

Example 2: Input: 10 → Output: Not Prime

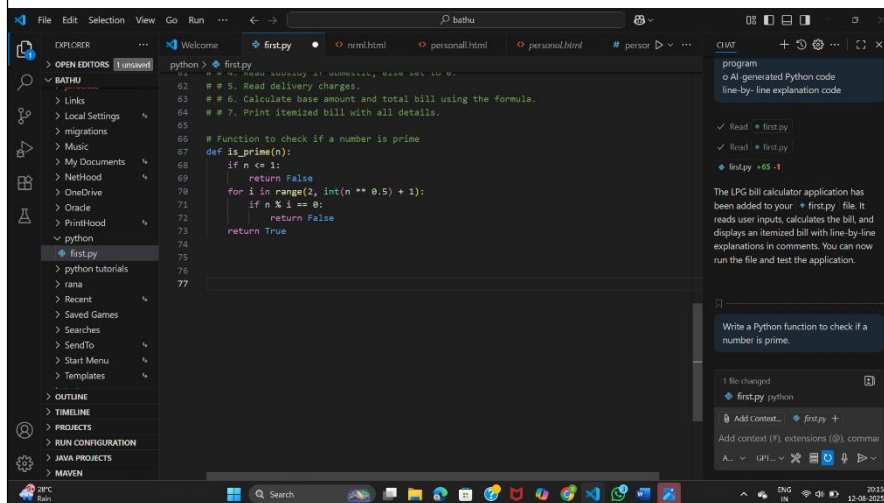
Example 3: Input: 2 → Output: Prime

Generate the function accordingly.

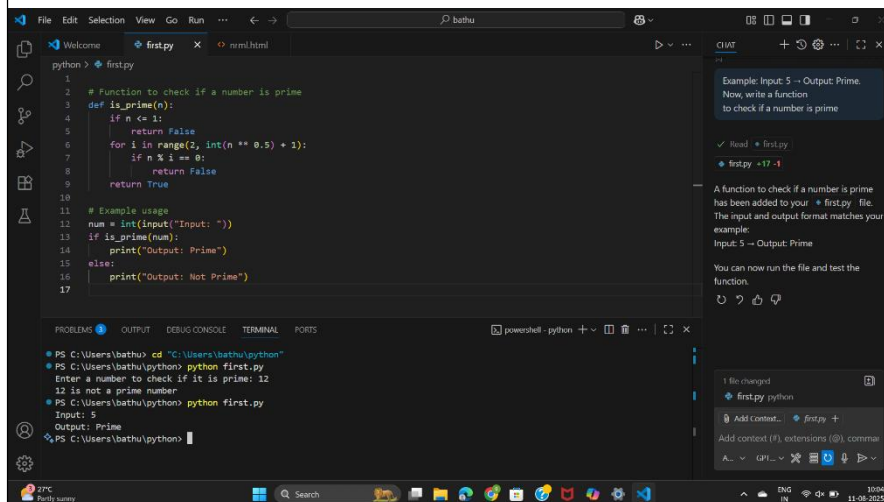
→ Context-Managed (With Optimization)

Vs code github-copilot:

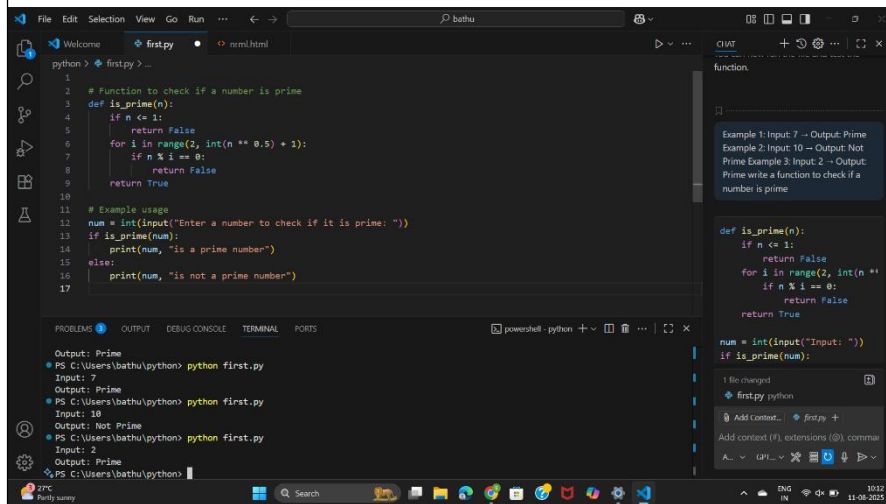
→ Zero-shot:



→ One-shot:



➔ Few-shots:

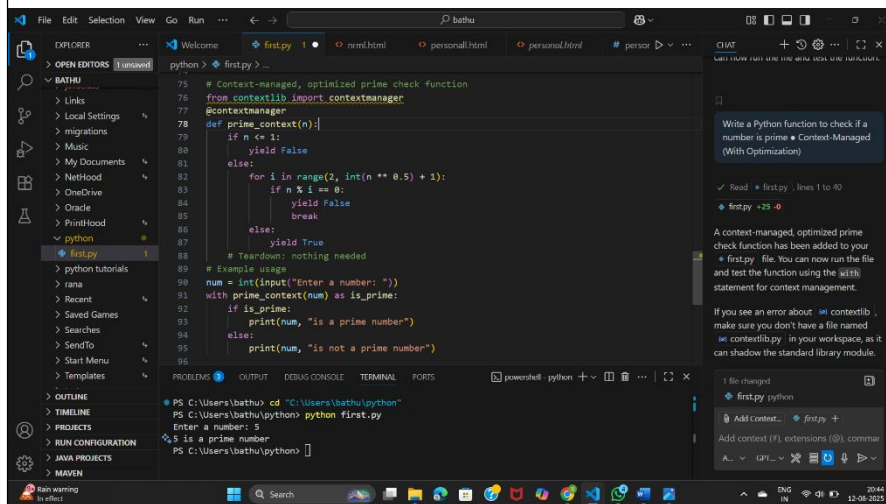


The screenshot shows the VS Code editor with a file named `first.py`. The code defines a function `is_prime(n)` that checks if a number is prime. It includes a docstring with examples: "Example 1: Input: 7 -> Output: Prime", "Example 2: Input: 10 -> Output: Not Prime", and "Example 3: Input: 2 -> Output: Prime". The function uses a loop to check divisibility from 2 to $\sqrt{n}$. The terminal shows the execution of the script, which prompts the user to enter a number and prints the result. The output shows that 7 is prime, 10 is not prime, and 2 is prime.

```
python> python first.py
1
2 # Function to check if a number is prime
3 def is_prime(n):
4     if n <= 1:
5         return False
6     for i in range(2, int(n ** 0.5) + 1):
7         if n % i == 0:
8             return False
9     return True
10
11 # Example usage
12 num = int(input("Enter a number to check if it is prime: "))
13 if is_prime(num):
14     print(num, "is a prime number")
15 else:
16     print(num, "is not a prime number")
17
```

Output: Prime
Input: 7
Output: Prime
Input: 10
Output: Not Prime
Input: 2
Output: Prime

➔ Context-Managed (With Optimization):



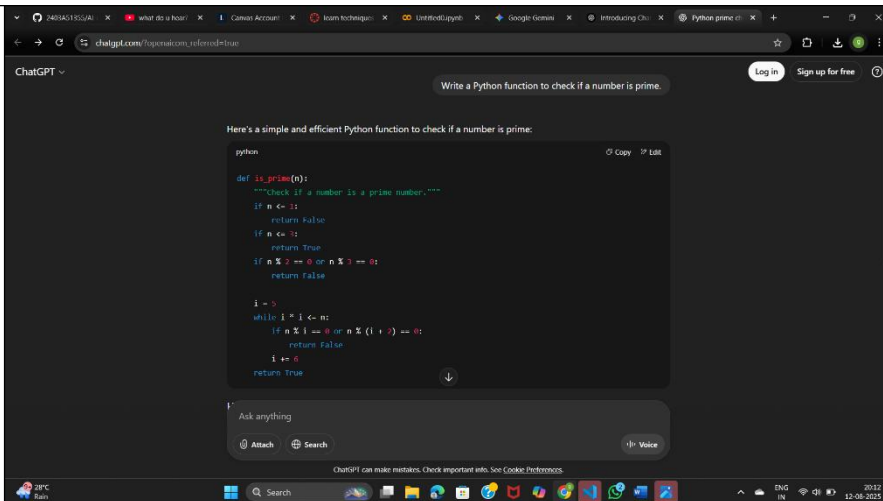
The screenshot shows the VS Code editor with a file named `first.py`. The code defines a function `prime_context(n)` that checks if a number is prime. It uses a context manager to optimize the function. The function uses a loop to check divisibility from 2 to $\sqrt{n}$. The terminal shows the execution of the script, which prompts the user to enter a number and prints the result. The output shows that 5 is prime.

```
python> python first.py
75 # Context-managed, optimized prime check function
76 from contextlib import contextmanager
77 @contextmanager
78 def prime_context(n):
79     if n <= 1:
80         yield False
81     else:
82         for i in range(2, int(n ** 0.5) + 1):
83             if n % i == 0:
84                 yield False
85                 break
86             else:
87                 yield True
88 # Example usage
89 num = int(input("Enter a number: "))
90 with prime_context(num) as is_prime:
91     if is_prime:
92         print(num, "is a prime number")
93     else:
94         print(num, "is not a prime number")
95
```

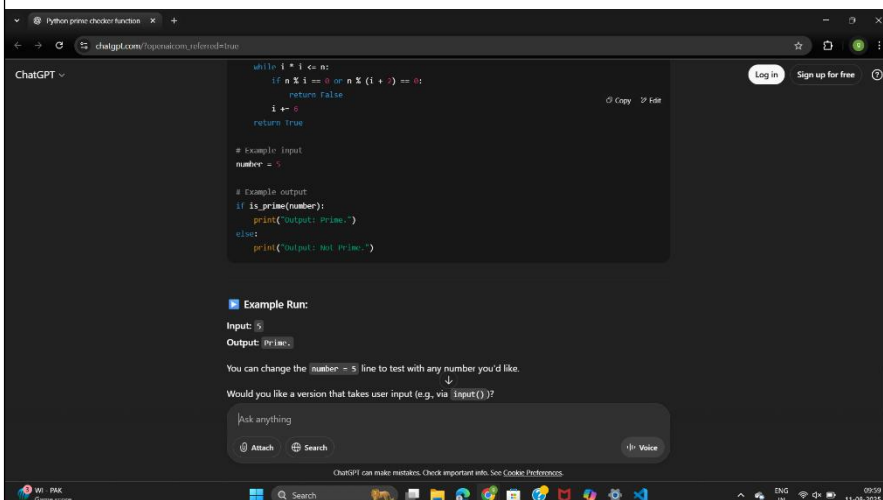
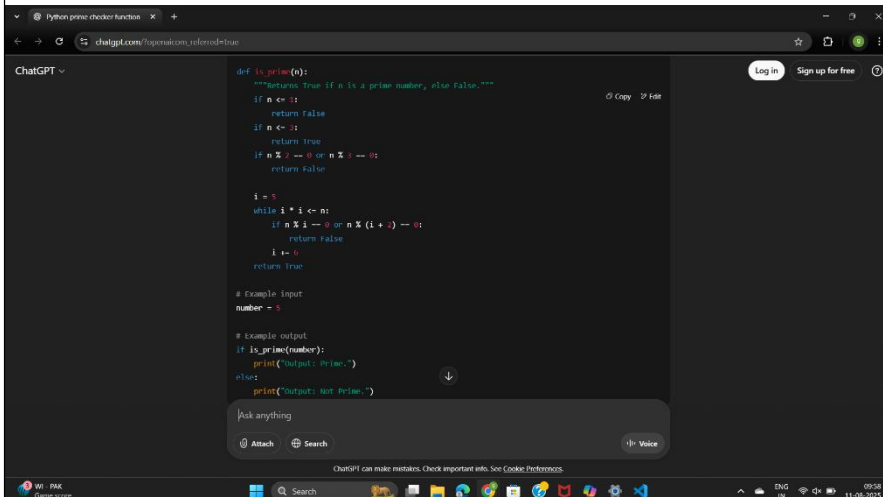
Output: 5 is a prime number

chatGpt:

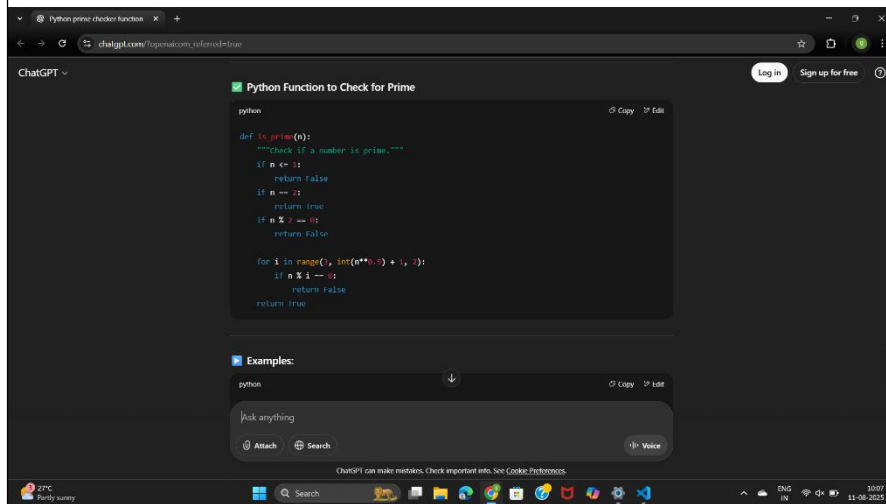
➔ Zero-shot:



➔ One-shot:



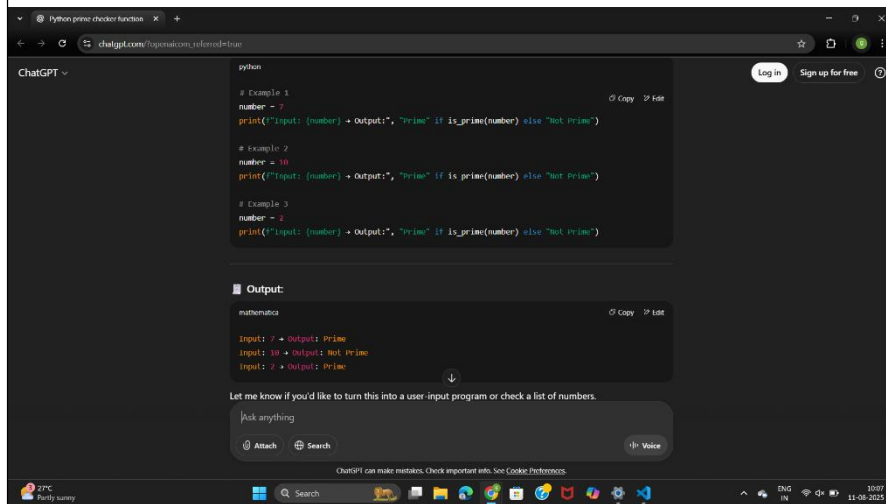
➔ Few-shots:



The screenshot shows the ChatGPT web interface. The title bar indicates the browser is 'Python prime checker function'. The address bar shows 'chatgpt.com/openai.com/indexed.html'. The ChatGPT logo is in the top left. The main content area displays a Python function titled 'Python Function to Check for Prime'. The function is as follows:

```
python
def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    if n <= 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True
```

Below the function, there is an 'Examples:' section with a text input field containing 'Ask anything'. There are also buttons for 'Attach', 'Search', and 'Voice'. The bottom status bar shows '27°C Partly sunny', a search bar, and system icons for language (ENG), time (11:07), and date (11-06-2025).



The screenshot shows the ChatGPT web interface. The title bar indicates the browser is 'Python prime checker function'. The address bar shows 'chatgpt.com/openai.com/indexed.html'. The ChatGPT logo is in the top left. The main content area displays a Python function titled 'Python Function to Check for Prime'. The function is as follows:

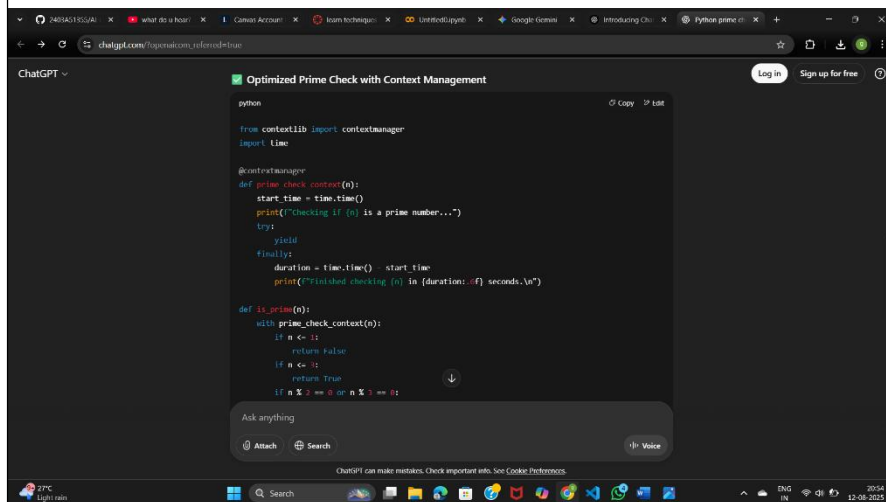
```
python
# Example 1
number = 7
print(f"Input: {number} -> Output: {'Prime' if is_prime(number) else 'Not Prime'}")

# Example 2
number = 10
print(f"Input: {number} -> Output: {'Prime' if is_prime(number) else 'Not Prime'}")

# Example 3
number = 2
print(f"Input: {number} -> Output: {'Prime' if is_prime(number) else 'Not Prime'}")
```

Below the function, there is an 'Output:' section with a text input field containing 'Ask anything'. There are also buttons for 'Attach', 'Search', and 'Voice'. The bottom status bar shows '27°C Partly sunny', a search bar, and system icons for language (ENG), time (11:07), and date (11-06-2025).

➔ Context-Managed (With Optimization):



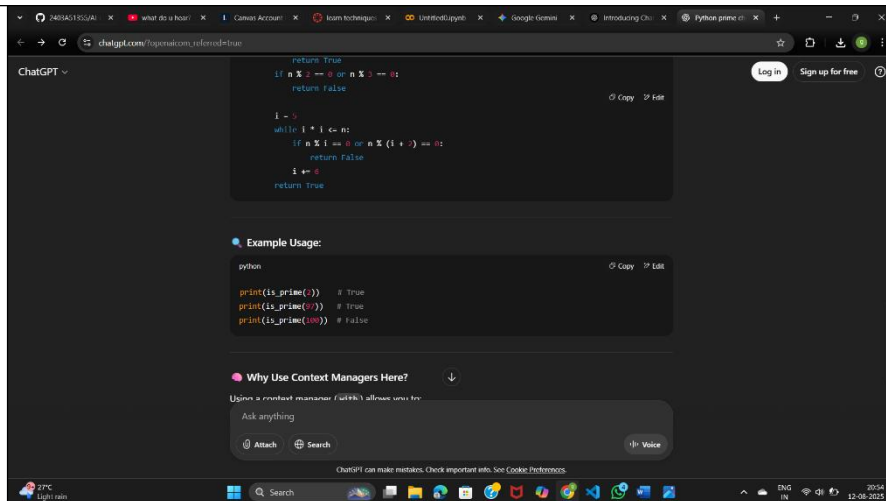
The screenshot shows the ChatGPT web interface. The title bar indicates the browser is 'Python prime checker function'. The address bar shows 'chatgpt.com/openai.com/indexed.html'. The ChatGPT logo is in the top left. The main content area displays a Python function titled 'Optimized Prime Check with Context Management'. The function is as follows:

```
python
from contextlib import contextmanager
import time

@contextmanager
def prime_check_context(n):
    start_time = time.time()
    print(f"Checking if {n} is a prime number...")
    try:
        yield
    finally:
        duration = time.time() - start_time
        print(f"Finished checking {n} in {duration:.6f} seconds.\n")

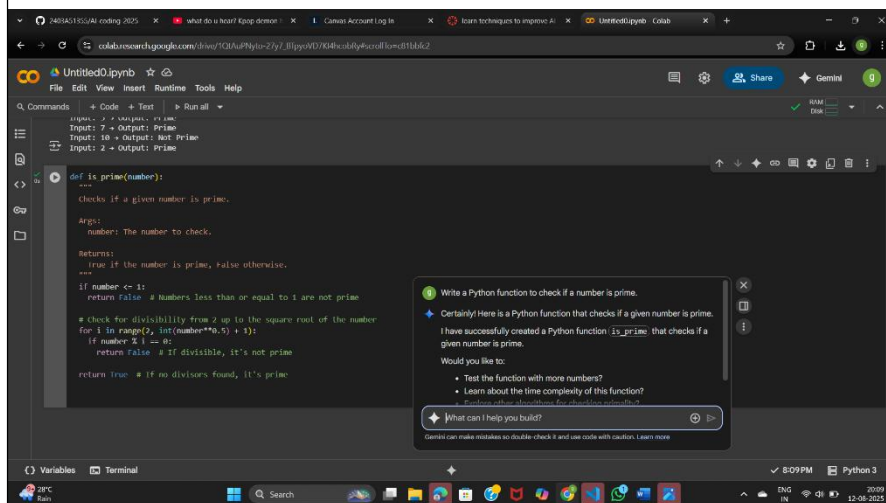
def is_prime(n):
    with prime_check_context(n):
        if n <= 1:
            return False
        if n <= 2:
            return True
        if n % 2 == 0 or n % 3 == 0:
```

Below the function, there is an 'Output:' section with a text input field containing 'Ask anything'. There are also buttons for 'Attach', 'Search', and 'Voice'. The bottom status bar shows '27°C Partly sunny', a search bar, and system icons for language (ENG), time (12:04), and date (12-06-2025).



Google Colab:

→ Zero-shot:



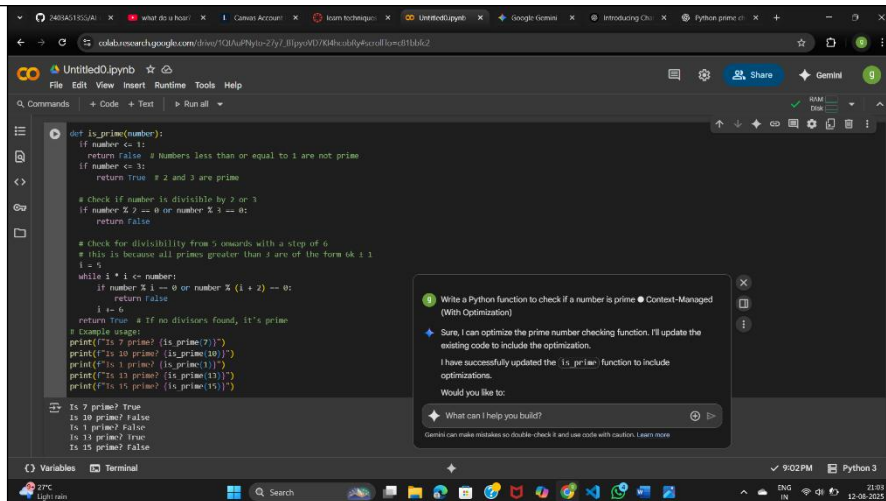
→ One-shot:

The screenshot shows a Jupyter Notebook titled 'Untitled0.ipynb' in a web browser. The code cell contains a Python function `def is_prime(number):` that checks if a number is prime. It includes comments explaining the logic: checking for divisibility from 2 up to the square root of the number. Below the function, there's an example usage: `print("Area of rectangle with length 5 and width 10: (calculate_rectangle_area(5, 10))")`. A chat window on the right shows a conversation with Gemini AI. The user asks: "Example Input: 5 -> Output: Prime. Now, write a function to check if a number is prime". Gemini responds: "I have already provided a function to check if a number is prime. I'll add an example using the input and output format you provided. I've added the example with the specified input/output format to the code cell. Is there anything else you would like to do? Perhaps you'd like to test this function with more numbers or explore other number-related functions?".

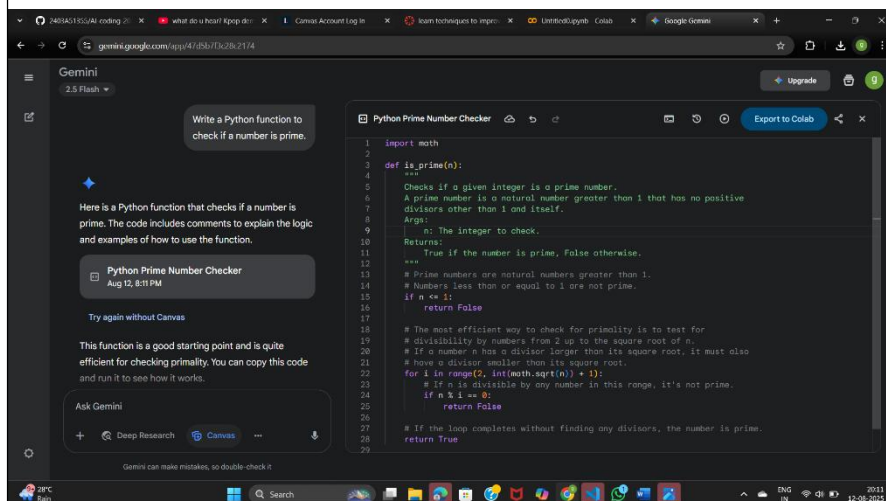
➔ Few-shot:

The screenshot shows the same Jupyter Notebook interface, but now with multiple examples of the `is_prime` function being tested. The code cell includes several lines of code that call `is_prime` with different inputs and print the results in a specific format. For example: `input_number = 5`, `output = "Prime" if is_prime(input_number) else "Not Prime"`, and `print(f"Input: {input_number} > Output: {output}")`. A chat window on the right shows a conversation with Gemini AI. The user asks: "I understand you'd like more examples of the prime number function. I've added your examples to the code cell. I've added all the examples you provided to the code cell and verified that the function works for them. Do you have any other numbers you'd like to test, or would you like to explore other mathematical functions?".

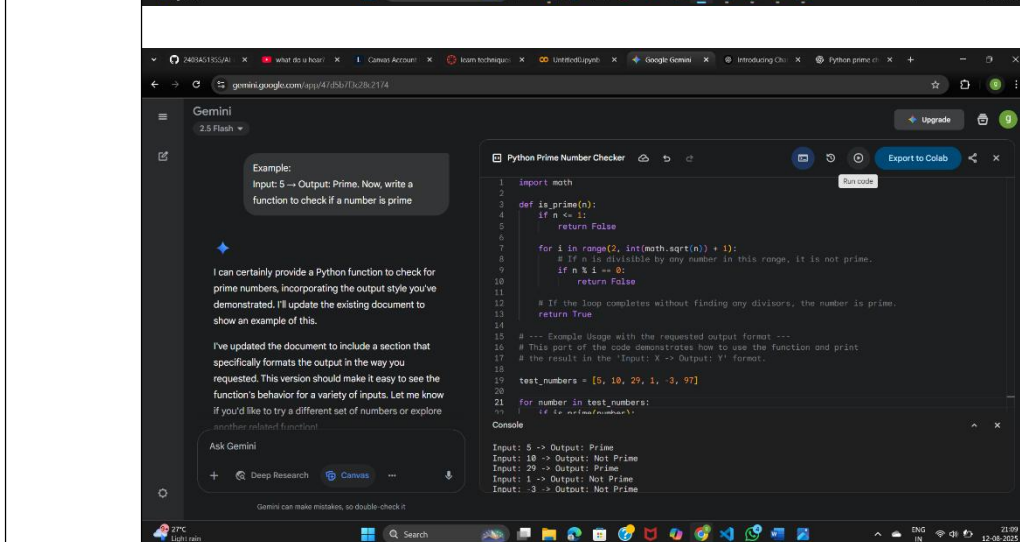
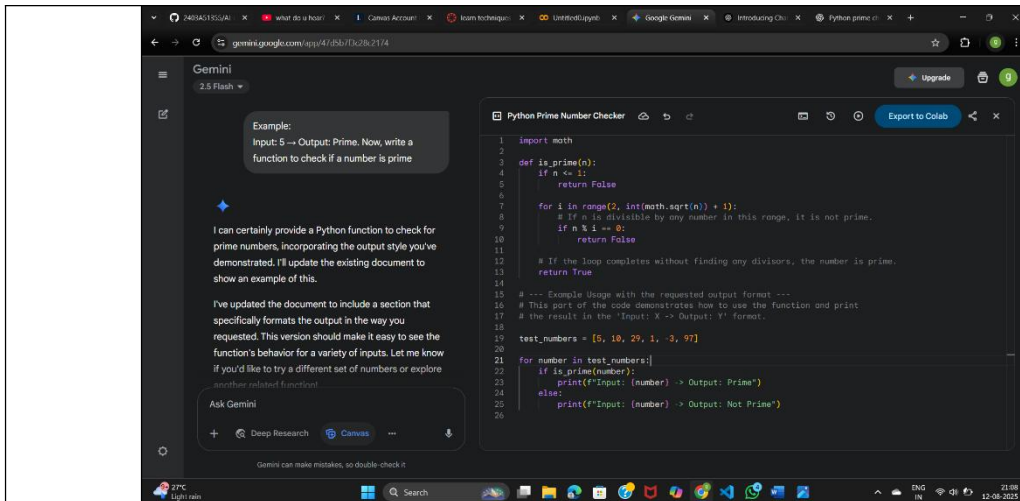
➔ Context-Managed (With Optimization):



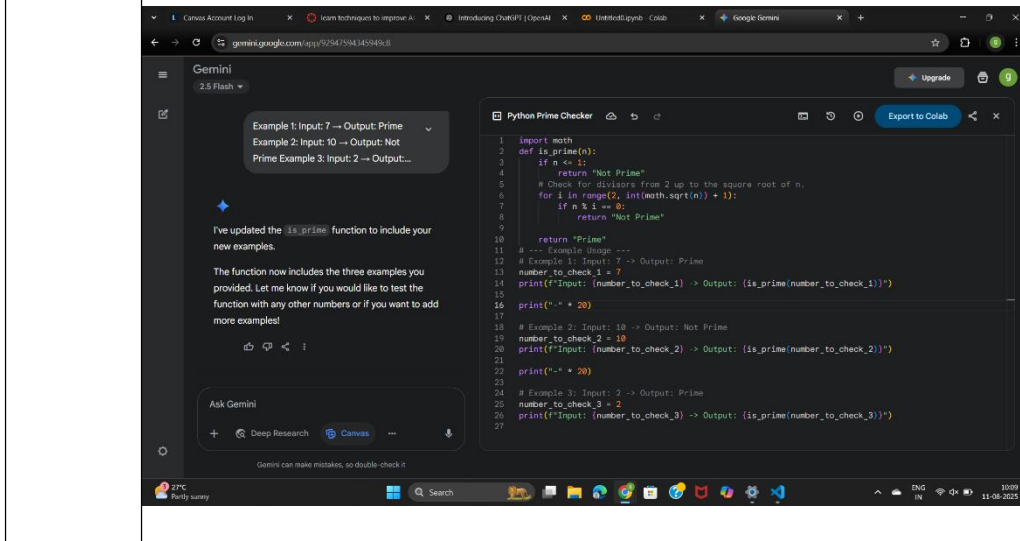
Google Gemini: ➔ Zero-shot:



➔ One-shot:



➔ Few-shot:



Canvas Account Log in

Introducing ChatGPT | OpenAI

Google Gemini

Gemini

2.5 Flash

Example 1: Input: 7 → Output: Prime
Example 2: Input: 10 → Output: Not Prime
Prime Example 3: Input: 2 → Output: Prime

I've updated the `is_prime` function to include your new examples.

The function now includes the three examples you provided. Let me know if you would like to test the function with any other numbers or if you want to add more examples!

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

Python Prime Checker

```
1 import math
2 def is_prime(n):
3     if n <= 1:
4         return "Not Prime"
5     # Check for divisors from 2 up to the square root of n.
6     for i in range(2, int(math.sqrt(n)) + 1):
7         if n % i == 0:
8             return "Not Prime"
9     return "Prime"
10
11 # --- Example Usage ---
12 # Example 1: Input: 7 -> Output: Prime
13 number_to_check_1 = 7
14 print(f"Input: {number_to_check_1} -> Output: {is_prime(number_to_check_1)}")
15
16 print("-" * 20)
17
18 # Example 2: Input: 10 -> Output: Not Prime
19 number_to_check_2 = 10
20 print(f"Input: {number_to_check_2} -> Output: {is_prime(number_to_check_2)}")
21
22 output = "-" * 20
```

Console

Input: 7 -> Output: Prime

Input: 10 -> Output: Not Prime

Input: 2 -> Output: Prime

23°C
Partly sunny

Search

100%
11:09
11-08-2025

→ Context-Managed (With Optimization):

Gemini

2.5 Flash

Write a Python function to check if a number is prime • Context-Managed (With Optimization)

That's a great request. While a simple primality check function doesn't typically require a Python context manager, I believe you're looking for a version of the code that is more efficient and "context-managed" in a broader sense of being a robust and well-optimized solution.

Here is an updated Python function that includes several classic optimizations to make the primality check significantly faster, especially for larger numbers.

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

Optimized Python Prime Number Checker

```
1 import math
2
3 def is_prime_optimized(n):
4     """
5     Checks if a given integer is a prime number using optimizations.
6
7     This function is more efficient than a basic implementation by:
8     1. Handling the special cases of 1, 2, and 3 upfront.
9     2. Immediately eliminating all even numbers greater than 2.
10    3. Checking for divisibility only by numbers of the form 6k +/- 1,
11       which greatly reduces the number of checks needed.
12
13    Args:
14        n: The integer to check.
15
16    Returns:
17        True if the number is prime, False otherwise.
18    """
19    # 1. Numbers less than or equal to 1 are not prime.
20    if n <= 1:
21        return False
22
23    # 2. Handle the first few prime numbers as special cases.
24    # 2 and 3 are prime.
25    if n <= 3:
26        return True
27
28    if n % 2 == 0 or n % 3 == 0:
29        return False
```

24°C
Rain warning
in effect

Search

21°C
12-08-2025

Gemini

2.5 Flash

Write a Python function to check if a number is prime • Context-Managed (With Optimization)

That's a great request. While a simple primality check function doesn't typically require a Python context manager, I believe you're looking for a version of the code that is more efficient and "context-managed" in a broader sense of being a robust and well-optimized solution.

Here is an updated Python function that includes several classic optimizations to make the primality check significantly faster, especially for larger numbers.

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

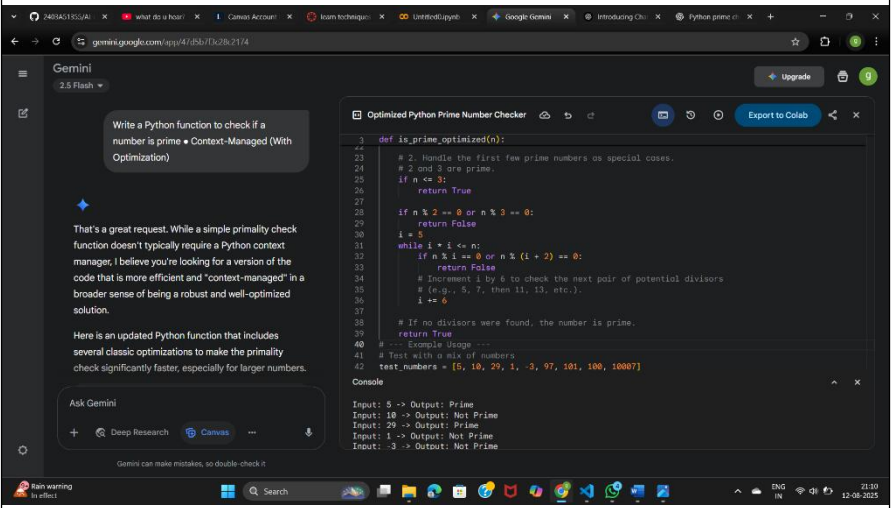
Optimized Python Prime Number Checker

```
3 def is_prime_optimized(n):
4     """
5     Checks if a given integer is a prime number using optimizations.
6
7     This function is more efficient than a basic implementation by:
8     1. Handling the special cases of 1, 2, and 3 upfront.
9     2. Immediately eliminating all even numbers greater than 2.
10    3. Checking for divisibility only by numbers of the form 6k +/- 1,
11       which greatly reduces the number of checks needed.
12
13    Args:
14        n: The integer to check.
15
16    Returns:
17        True if the number is prime, False otherwise.
18    """
19    # 1. Numbers less than or equal to 1 are not prime.
20    if n <= 1:
21        return False
22
23    # 2. Handle the first few prime numbers as special cases.
24    # 2 and 3 are prime.
25    if n <= 3:
26        return True
27
28    if n % 2 == 0 or n % 3 == 0:
29        return False
30
31    i = 5
32    while i * i <= n:
33        if n % i == 0 or n % (i + 2) == 0:
34            return False
35        # Increment i by 6 to check the next pair of potential divisors
36        # (e.g., 5, 7, then 11, 13, etc.).
37        i += 6
38
39    # If no divisors were found, the number is prime.
40    return True
41
42 # --- Example Usage ---
43 # Test with a mix of numbers
44 test_numbers = [5, 10, 29, 1, -3, 97, 101, 100, 10007]
45
46 for number in test_numbers:
47     if is_prime_optimized(number):
48         print(f"Input: {number} -> Output: Prime")
49     else:
50         print(f"Input: {number} -> Output: Not Prime")
```

24°C
Rain warning
in effect

Search

21°C
12-08-2025

	 The screenshot shows the Google Gemini AI web interface. On the left, a chat window contains a prompt: "Write a Python function to check if a number is prime • Context-Managed (With Optimization)". The AI's response explains that a simple primality check doesn't typically require a Python context manager and provides an updated, optimized Python function. On the right, a code editor titled "Optimized Python Prime Number Checker" displays the function code. Below the code, a console window shows the function being tested with various inputs and their corresponding outputs. <pre>def is_prime_optimized(n): # 2 handles the first few prime numbers as special cases. # 2 and 3 are prime. if n <= 3: return True if n % 2 == 0 or n % 3 == 0: return False i = 5 while i * i <= n: if n % i == 0 or n % (i + 2) == 0: return False # Increment i by 6 to check the next pair of potential divisors # (e.g., 5, 7, then 11, 13, etc.). i += 6 # If no divisors were found, the number is prime. return True # --- Sample Usage --- # Test with a mix of numbers test_numbers = [5, 10, 29, 1, -3, 97, 101, 100, 10007] for n in test_numbers: print(f"Input: {n} -> Output: {'Prime' if is_prime_optimized(n) else 'Not Prime'}")</pre> Console Output: Input: 5 -> Output: Prime Input: 10 -> Output: Not Prime Input: 29 -> Output: Prime Input: 1 -> Output: Not Prime Input: -3 -> Output: Not Prime	
2	Task: Mobile Data Usage Billing Application (1.0 Marks) Objective: Use Python programming and AI-assisted coding tools to create an application that simulates mobile data billing for a telecom service provider. Instructions 1. Use GitHub Copilot or Google Gemini to assist in writing the program. 2. Read the following inputs from the user: ○ Data Consumed (in GB) ○ Plan Type (Prepaid / Postpaid) ○ Additional Services Used (e.g., caller tune, OTT subscription, etc.) 3. Implement billing logic to calculate: ○ DC (Data Charges) – charges based on data consumption ○ VC (Value-added Charges) – charges for additional services ○ Tax – applicable tax on the total bill 4. Display an itemized bill showing: ○ Plan Type ○ Data Usage and Charges ○ Value-added Services and Charges	Week2 - Monday

- Tax
- Total Bill Amount

Requirements

- Students must refer to their actual mobile bill for charge structure (data cost, service fees, taxes) to make the program realistic.
- AI assistance (Copilot/Gemini) must be used to generate and refine the initial code.

Deliverables

- AI prompts used for code generation.
- AI-generated Python code and any optimized version.
- Screenshots of:
 - AI interactions
 - Program execution and output
 - Comparison with the student's actual mobile bill.

➔ Github copilot:

```

python > first.py
1 # Mobile Data Usage Billing Application
2 # This program simulates mobile data billing for a telecom service provider.
3
4 # Step 1: Read user inputs
5 plan_type = input("Enter Plan Type (Prepaid/Postpaid): ")
6 data_consumed = float(input("Enter Data Consumed (in GB): "))
7 services = input("Enter Additional Services Used (comma separated, e.g., caller tune, OTT): ")
8
9 # Step 2: Define realistic charges (example values, adjust to match your actual bill)
10 data_charge_per_gb = 10.0 if plan_type.lower() == 'prepaid' else 8.0 # Prepaid: ₹10/GB, Postpaid: ₹8/GB
11 service_charges = {
12     'caller tune': 30.0,
13     'OTT': 100.0,
14     'international roaming': 200.0,
15     'sms pack': 20.0
16 }
17
18 # Step 3: Calculate charges
19 DC = data_consumed * data_charge_per_gb # Data Charges
20 VC = sum(service_charges.get(s.strip().lower(), 0.0) for s in services if s.strip()) # Value-added Services
21 subtotal = DC + VC
22 Tax = subtotal * 0.18 # 18% GST
23 Total = subtotal + Tax
24
25 # Step 4: Display itemized bill
26 print("\n--- Mobile Data Usage Bill ---")
27 print(f"Plan Type: {plan_type}")
28 print(f>Data Used: {data_consumed} GB")
29 print(f>Data Charges (DC): ₹{DC:.2f}")
30 print(f"Value-added Services: {', '.join([s.strip() for s in services if s.strip()])} if VC applied")
31 print(f"Total Charges (Total): ₹{Total:.2f}")
  
```



```
python > first.py
21 subtotal = DC + VC
22 Tax = subtotal * 0.18 # 18% GST
23 Total = subtotal + Tax
24
25 # Step 4: Display itemized bill
26 print("\n--- Mobile Data Usage Bill ---")
27 print(f"Plan Type: {plan_type}")
28 print(f"Data Charges (DC): {(DC:.2f)}")
29 print(f"Value-added Services (V): {' '.join([s.strip() for s in services if s.strip()]) if VC else ''}")
30 print(f"Value-added Charges (VC): {(VC:.2f)}")
31 print(f"Tax (18% GST): {(Tax:.2f)}")
32 print(f"Total Bill Amount: {(Total:.2f)}")
33
34
35 # Line-by-line explanation:
36 # 1. Read plan type, data consumed, and additional services from user.
37 # 2. Set data charge per GB based on plan type and define service charges.
38 # 3. Calculate data charges, value-added charges, subtotal, tax, and total.
39 # 4. Print an itemized bill with all details.
40
```

```
PS C:\Users\bathu\python> python first.py
Enter a number: 5
5 is a prime number
PS C:\Users\bathu\python> python first.py
Enter Plan Type (Prepaid/Postpaid): prepaid
Enter Data Consumed (in GB): 2
Enter Additional Services Used (comma separated, e.g., caller tune, OTT): caller tune

--- Mobile Data Usage Bill ---
Plan Type: prepaid
Data Used: 2.0 GB
Data Charges (DC): ₹20.00
Value-added Services: caller tune
Value-added Charges (VC): ₹30.00
Tax (18% GST): ₹9.00
Total Bill Amount: ₹59.00
PS C:\Users\bathu\python>
```

➔ Google Gemini:

Enter data consumed (in GB):
Traceback (most recent call last):
File "string.py", line 23, in <module>...

Hello. Thank you for pointing that out. The `EOFError` you're seeing occurs when the program tries to get input in a non-interactive environment where there is no user to type. It's a common issue when running code in some online platforms.

I have fixed the code by adding a `try...except` block to handle this specific error. Now, if the program cannot read from standard input, it will use some default values to demonstrate its functionality instead of crashing. This makes the code more robust and allows it to run.

```
Initial Mobile Data Billing Application
1 #
2 # Mobile Data Billing Application - Initial Version (FIXED for EOFError)
3 # This program calculates a mobile bill based on user inputs.
4 #
5 import sys
6
7 # --- Define Rates (These can be updated based on your actual bill) ---
8 # Data charges are simplified for this example.
9 POSTPAID_DATA_RATE = 10 # Cost per GB
10 PREPAID_DATA_RATE = 12 # Cost per GB
11
12 # Value-added services and their costs
13 SERVICE_RATES = {
14     "caller tune": 60,
15     "ott subscription": 150,
16     "none": 0 # To handle cases where no extra services are used
17 }
18
19 TAX_RATE = 0.12 # 12% tax
20
21 # --- Get User Inputs ---
22 try:
23     # Attempt to get interactive input
24     data_consumed_gb = float(input("Enter data consumed (in GB): "))
25     plan_type = ""
26     while plan_type not in ["prepaid", "postpaid"]:
27         plan_type = input("Enter plan type (Prepaid / Postpaid): ").lower()
28         if plan_type not in ["prepaid", "postpaid"]:
29             print("Invalid plan type. Please choose 'Prepaid' or 'Postpaid'.")
30
```

240343155JA

what do a hour

Canvas Account

learn technologies

Untitled:ipynb

Google Gemini

Introducing Ch...

Python prime

gemini.google.com/app/47/6bu/Tk2Bz21/4

Gemini

2.5 Flash

Upgrade

0

Enter data consumed (in GB):

Traceback (most recent call last):

File "string", line 23, in <modu...

Hello. Thank you for pointing that out. The `EOFError` you're seeing occurs when the program tries to get input in a non-interactive environment where there is no user to type. It's a common issue when running code in some online platforms.

I have fixed the code by adding a `try...except` block to handle this specific error. Now, if the program cannot read from standard input, it will use some default values to demonstrate its functionality instead of crashing. This makes the code more robust and allows it to run.

Ask Gemini

Deep Research

Canvas

Gemini can make mistakes, so double-check it

Initial Mobile Data Billing Application

Export to Colab

```
22 try:
23     while plan_type not in ["prepaid", "postpaid"]:
24         additional_services_input = input("Enter additional services used (e.g., 'caller time, ott subscri
31
32 except (ValueError, EOFError):
33     # Handle the cases where input is invalid or not available (like in a non-interactive terminal)
34     print("\n--- Non-interactive session detected or invalid input. Using default values. ---")
35     data_consumed_gb = 15.5
36     plan_type = "postpaid"
37     additional_services_input = "caller time, ott subscription"
38
39 # --- Implement Billing Logic ---
40
41 # 1. Calculate Data Charges (DC)
42 data_charges = 0
43 if plan_type == "prepaid":
44     data_charges = data_consumed_gb * PREPAID_DATA_RATE
45 else: # It must be "postpaid" due to the while loop validation
46     data_charges = data_consumed_gb * POSTPAID_DATA_RATE
47
48 # 2. Calculate Value-added Charges (VC)
49 value_added_charges = 0
50 services_used_list = [s.strip() for s in additional_services_input.split(',')]
51 for service in services_used_list:
52     if service in SERVICE_RATES:
53         value_added_charges += SERVICE_RATES[service]
54     elif service != "none":
55         print(f"Warning: '{service}' is not a valid service. No charge applied.")
56
57
58 # 3. Calculate Subtotal and Tax
59 subtotal = data_charges + value_added_charges
60 tax = subtotal * TAX_RATE
61
62 # 4. Calculate Total Bill Amount
63 total_bill = subtotal + tax
64
65 # --- Display Itemized Bill ---
66 print("\n--- Mobile Bill Summary ---")
67 print(f"Plan Type: {plan_type.capitalize()}")
68 print(f"Data Usage: {data_consumed_gb:.2f} GB")
69 print(f"Data Charges (DC): Rs. {data_charges:.2f}")
70
71 # Display a list of services and their charges
72 print("Value-added Services:")
73 for service in services_used_list:
74     if service in SERVICE_RATES:
75         print(f"- {service.capitalize():15} Rs. {SERVICE_RATES[service]:.2f}")
76
77
78 # 5. Final Bill Summary
79 print(f"Subtotal: Rs. {subtotal:.2f}")
80 print(f"Tax (12%): Rs. {tax:.2f}")
81 print(f"Total Bill Amount: Rs. {total_bill:.2f}")
82
83 """ Code execution complete """
84 Enter data consumed (in GB):
85 --- Non-interactive session detected or invalid input. Using default values. ---
86
87 --- Mobile Bill Summary ---
88 Plan Type: Postpaid
89 Data Usage: 15.50 GB
90 Data Charges (DC): Rs. 155.00
91 Value-added Services:
92 - Caller time: Rs. 50.00
93 - Ott subscription: Rs. 100.00
94 Value-added Charges (VC): Rs. 200.00
95 Subtotal: Rs. 355.00
96
97 Tax (12%): Rs. 42.60
98 Total Bill Amount: Rs. 397.60
99
100 """ Code execution complete """
101 Enter data consumed (in GB):
102 --- Non-interactive session detected or invalid input. Using default values. ---
103
104 --- Mobile Bill Summary ---
105 Plan Type: Postpaid
106 Data Usage: 15.50 GB
107 Data Charges (DC): Rs. 155.00
108 Value-added Services:
109 - Caller time: Rs. 50.00
110 - Ott subscription: Rs. 100.00
111 Value-added Charges (VC): Rs. 200.00
112 Subtotal: Rs. 355.00
113
114 Tax (12%): Rs. 42.60
115 Total Bill Amount: Rs. 397.60
116
117 """ Code execution complete """
```

27°C

Rain showers

Search

21:34

12-08-2025

240343155JA

what do a hour

Canvas Account

learn technologies

Untitled:ipynb

Google Gemini

Introducing Ch...

Python prime

gemini.google.com/app/47/6bu/Tk2Bz21/4

Gemini

2.5 Flash

Upgrade

0

Enter data consumed (in GB):

Traceback (most recent call last):

File "string", line 23, in <modu...

Hello. Thank you for pointing that out. The `EOFError` you're seeing occurs when the program tries to get input in a non-interactive environment where there is no user to type. It's a common issue when running code in some online platforms.

I have fixed the code by adding a `try...except` block to handle this specific error. Now, if the program cannot read from standard input, it will use some default values to demonstrate its functionality instead of crashing. This makes the code more robust and allows it to run.

Ask Gemini

Deep Research

Canvas

Gemini can make mistakes, so double-check it

Initial Mobile Data Billing Application

Export to Colab

```
48 # 2. Calculate Value added Charges (VC)
49 value_added_charges = 0
50 services_used_list = [s.strip() for s in additional_services_input.split(',')]
51 for service in services_used_list:
52     if service in SERVICE_RATES:
53         value_added_charges += SERVICE_RATES[service]
54     elif service != "none":
55         print(f"Warning: '{service}' is not a valid service. No charge applied.")
56
57
58 # 3. Calculate Subtotal and Tax
59 subtotal = data_charges + value_added_charges
60 tax = subtotal * TAX_RATE
61
62 # 4. Calculate Total Bill Amount
63 total_bill = subtotal + tax
64
65 # --- Display Itemized Bill ---
66 print("\n--- Mobile Bill Summary ---")
67 print(f"Plan Type: {plan_type.capitalize()}")
68 print(f"Data Usage: {data_consumed_gb:.2f} GB")
69 print(f"Data Charges (DC): Rs. {data_charges:.2f}")
70
71 # Display a list of services and their charges
72 print("Value-added Services:")
73 for service in services_used_list:
74     if service in SERVICE_RATES:
75         print(f"- {service.capitalize():15} Rs. {SERVICE_RATES[service]:.2f}")
76
77
78 # 5. Final Bill Summary
79 print(f"Subtotal: Rs. {subtotal:.2f}")
80 print(f"Tax (12%): Rs. {tax:.2f}")
81 print(f"Total Bill Amount: Rs. {total_bill:.2f}")
82
83 """ Code execution complete """
84 Enter data consumed (in GB):
85 --- Non-interactive session detected or invalid input. Using default values. ---
86
87 --- Mobile Bill Summary ---
88 Plan Type: Postpaid
89 Data Usage: 15.50 GB
90 Data Charges (DC): Rs. 155.00
91 Value-added Services:
92 - Caller time: Rs. 50.00
93 - Ott subscription: Rs. 100.00
94 Value-added Charges (VC): Rs. 200.00
95 Subtotal: Rs. 355.00
96
97 Tax (12%): Rs. 42.60
98 Total Bill Amount: Rs. 397.60
99
100 """ Code execution complete """
```

27°C

Rain showers

Search

21:34

12-08-2025

240343155JA

what do a hour

Canvas Account

learn technologies

Untitled:ipynb

Google Gemini

Introducing Ch...

Python prime

gemini.google.com/app/47/6bu/Tk2Bz21/4

Gemini

2.5 Flash

Upgrade

0

Input in a non-interactive environment where there is no user to type. It's a common issue when running code in some online platforms.

I have fixed the code by adding a `try...except` block to handle this specific error. Now, if the program cannot read from standard input, it will use some default values to demonstrate its functionality instead of crashing. This makes the code more robust and allows it to run correctly even in environments that don't support live user input.

Here is the updated version of the code:

The new code should now run successfully and provide a sample output without requiring interactive input.

Ask Gemini

Deep Research

Canvas

Gemini can make mistakes, so double-check it

Initial Mobile Data Billing Application

Export to Colab

Console

Enter data consumed (in GB):

--- Non-interactive session detected or invalid input. Using default values. ---

--- Mobile Bill Summary ---

Plan Type: Postpaid

Data Usage: 15.50 GB

Data Charges (DC): Rs. 155.00

Value-added Services:

- Caller time: Rs. 50.00

- Ott subscription: Rs. 100.00

Value-added Charges (VC): Rs. 200.00

Subtotal: Rs. 355.00

Tax (12%): Rs. 42.60

Total Bill Amount: Rs. 397.60

==== Code execution complete ===

Enter data consumed (in GB):

--- Non-interactive session detected or invalid input. Using default values. ---

--- Mobile Bill Summary ---

Plan Type: Postpaid

Data Usage: 15.50 GB

Data Charges (DC): Rs. 155.00

Value-added Services:

- Caller time: Rs. 50.00

- Ott subscription: Rs. 100.00

Value-added Charges (VC): Rs. 200.00

Subtotal: Rs. 355.00

Tax (12%): Rs. 42.60

Total Bill Amount: Rs. 397.60

==== Code execution complete ===

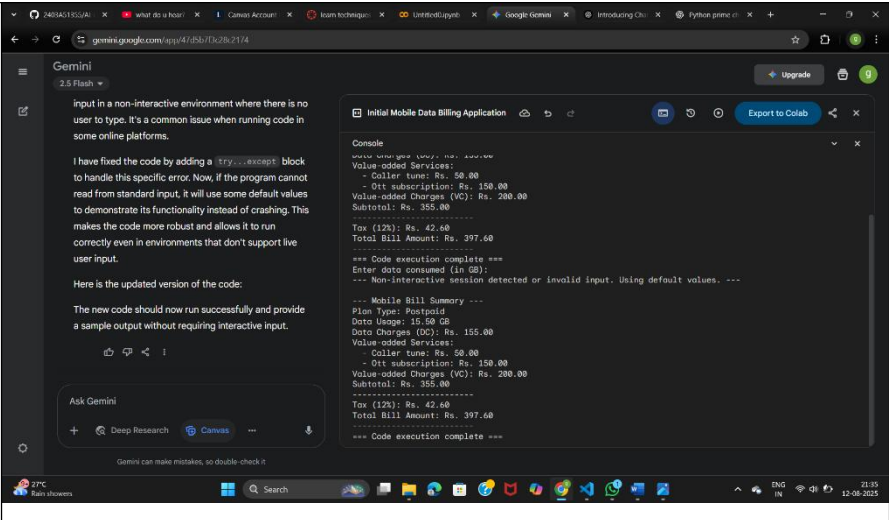
27°C

Rain showers

Search

21:35

12-08-2025

	 The screenshot shows the Google Gemini AI interface. On the left, there's a chat window with a message about handling errors in a non-interactive environment. The main area displays a Python code snippet for an 'Initial Mobile Data Billing Application'. The code calculates various charges like Collier tune, Dtt subscription, Value-added Charges (VC), Subtotal, and Tax (12%). The console output shows the results of the code execution, including a 'Mobile Bill Summary' with Plan Type: Postpaid, Data Usage: 15.50 GB, and a Total Bill Amount of Rs. 397.60.	
3	Task: Develop an LPG Billing System (1.0 Marks) Objective Apply your Python programming skills and utilize AI-assisted coding tools to build an application that calculates the LPG bill based on specified customer inputs and billing parameters. Instructions 1. Use GitHub Copilot or Google Gemini to assist in writing and refining the program. 2. Read the following user inputs: ○ Cylinder Type (Domestic 14.2 kg / Domestic 5 kg / Commercial 19 kg / Commercial 47.5 kg) ○ Number of Cylinders Booked ○ Subsidy Amount (applicable only for domestic cylinders) 3. Refer to the given LPG Price List to determine the price per cylinder: ○ Domestic LPG (14.2 kg) → ₹905.00 ○ Domestic LPG (5 kg) → ₹335.50 ○ Commercial LPG (19 kg) → ₹1,886.50	Week2 - Monday

- Commercial LPG (47.5 kg) → ₹4,712.00
- Delivery Charges (₹10 to ₹50)

4. Implement the billing formula:

Bill Amount = (Price per Cylinder × Quantity) - Subsidy (if applicable) + Delivery Charges

5. Calculate and display an itemized bill including:

- Cylinder Type
- Number of Cylinders
- Base Amount
- Subsidy
- Delivery Charges
- Total Bill Amount

Deliverables

- A report containing:
 - AI prompts used to generate the program
 - AI-generated Python code
 - Line-by-line explanation of the code

➔ Gthub-copilot:

```

python > firstpy
1 # LPG Bill Calculator Application
2 # This program calculates the LPG bill based on user inputs and billing parameters.
3
4 # Step 1: Read user inputs
5 print("LPG Cylinder Types:")
6 print("1. Domestic 14.2 kg")
7 print("2. Domestic 5 kg")
8 print("3. Commercial 19 kg")
9 print("4. Commercial 47.5 kg")
10
11 cylinder_type_input = input("Enter Cylinder Type (1/2/3/4): ") # User selects cylinder type
12 num_cylinders = int(input("Enter Number of Cylinders Booked: ")) # User enters quantity
13
14 # Step 2: Set price per cylinder based on type
15 if cylinder_type_input == '1':
16     cylinder_type = "Domestic 14.2 kg"
17     price_per_cylinder = 985.00
18     is_domestic = True
19 elif cylinder_type_input == '2':
20     cylinder_type = "Domestic 5 kg"
21     price_per_cylinder = 335.50
22     is_domestic = True
23 elif cylinder_type_input == '3':
24     cylinder_type = "Commercial 19 kg"
25     price_per_cylinder = 1886.50
26     is_domestic = False
27 elif cylinder_type_input == '4':
28     cylinder_type = "Commercial 47.5 kg"
29     price_per_cylinder = 4712.00
30     is_domestic = False
31 else:

```

The LPG bill calculator application has been added to your • firstpy file. It reads user inputs, calculates the bill, and displays an itemized bill with line-by-line explanations in comments. You can now run the file and test the application.

```
File Edit Selection View Go Run ... bathu
python > firstpy > ...
27 elif cylinder_type_input == '4':
28     cylinder_type = "Commercial 47.5 kg"
29     price_per_cylinder = 4712.00
30     is_domestic = False
31 else:
32     print("Invalid cylinder type selected.")
33     exit()
34
35 # Step 3: Read subsidy amount (only for domestic)
36 if is_domestic:
37     subsidy = float(input("Enter Subsidy Amount: "))
38 else:
39     subsidy = 0.0
40
41 # Step 4: Read delivery charges
42 delivery_charges = float(input("Enter Delivery Charges (₹10 to ₹50): "))
43
44 # Step 5: Calculate bill components
45 base_amount = price_per_cylinder * num_cylinders # Base price for all cylinders
46 bill_amount = base_amount - subsidy + delivery_charges # Final bill formula
47
48 # Step 6: Display itemized bill
49 print("\n--- LPG Bill ---")
50 print(f"Cylinder Type: {cylinder_type}")
51 print(f"Number of Cylinders: {num_cylinders}")
52 print(f"Base Amount: ₹{base_amount:.2f}")
53 print(f"Subsidy: ₹{subsidy:.2f}")
54 print(f"Delivery Charges: ₹{delivery_charges:.2f}")
55 print(f"Total Bill Amount: ₹{bill_amount:.2f}")
56
57 # Line-by-line explanation:
```

```
File Edit Selection View Go Run ... bathu
python > firstpy > ...
41 # Step 4: Read delivery charges
42 delivery_charges = float(input("Enter Delivery Charges (₹10 to ₹50): "))
43
44 # Step 5: Calculate bill components
45 base_amount = price_per_cylinder * num_cylinders # Base price for all cylinders
46 bill_amount = base_amount - subsidy + delivery_charges # Final bill formula
47
48 # Step 6: Display itemized bill
49 print("\n--- LPG Bill ---")
50 print(f"Cylinder Type: {cylinder_type}")
51 print(f"Number of Cylinders: {num_cylinders}")
52 print(f"Base Amount: ₹{base_amount:.2f}")
53 print(f"Subsidy: ₹{subsidy:.2f}")
54 print(f"Delivery Charges: ₹{delivery_charges:.2f}")
55 print(f"Total Bill Amount: ₹{bill_amount:.2f}")
56
57 # Line-by-line explanation:
58 # 1. Display cylinder types and read user selection.
59 # 2. Read number of cylinders booked.
60 # 3. Set price per cylinder and check if domestic.
61 # 4. Read subsidy if domestic, else set to 0.
62 # 5. Read delivery charges.
63 # 6. Calculate base amount and total bill using the formula.
64 # 7. Print itemized bill with all details.
65
```

```
File Edit Selection View Go Run ... bathu
python > firstpy > ...
41 # Step 4: Read delivery charges
42 delivery_charges = float(input("Enter Delivery Charges (₹10 to ₹50): "))
43
44 # Step 5: Calculate bill components
45 base_amount = price_per_cylinder * num_cylinders # Base price for all cylinders
46 bill_amount = base_amount - subsidy + delivery_charges # Final bill formula
47
48 # Step 6: Display itemized bill
49 print("\n--- LPG Bill ---")
50 print(f"Cylinder Type: {cylinder_type}")
51 print(f"Number of Cylinders: {num_cylinders}")
52 print(f"Base Amount: ₹{base_amount:.2f}")
53 print(f"Subsidy: ₹{subsidy:.2f}")
54 print(f"Delivery Charges: ₹{delivery_charges:.2f}")
55 print(f"Total Bill Amount: ₹{bill_amount:.2f}")
56
57 # Line-by-line explanation:
58 # 1. Display cylinder types and read user selection.
59 # 2. Read number of cylinders booked.
60 # 3. Set price per cylinder and check if domestic.
61 # 4. Read subsidy if domestic, else set to 0.
62 # 5. Read delivery charges.
63 # 6. Calculate base amount and total bill using the formula.
64 # 7. Print itemized bill with all details.
65
```

→ Google Gemini:

Gemini 2.5 Flash

Hello! I can certainly help you with that. I've created a Python program that calculates the LPG bill according to all of your specified instructions. Below, you'll find a report outlining the process and the final, well-commented code.

LPG Bill Calculator Report Aug 11, 10:36 AM [Open](#)

LPG Bill Calculator Aug 11, 10:37 AM

I hope this application is exactly what you were looking for! The code is ready to be run as-is, but I've also included comments on how you could modify it to accept user input.

Let me know if you would like to make any changes.

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

```
1 import random
2
3 def calculate_lpg_bill(cylinder_type, num_cylinders, subsidy_amount):
4     # 3. Refer to the given LPG Price List to determine the price per cylinder.
5     lpg_price_list = {
6         "Domestic 14.2 kg": 905.00,
7         "Domestic 5 kg": 235.50,
8         "Commercial 19 kg": 1886.50,
9         "Commercial 47.5 kg": 4712.00
10    }
11
12    # Get the price per cylinder based on the provided type.
13    price_per_cylinder = lpg_price_list.get(cylinder_type, 0.0)
14
15    if price_per_cylinder == 0.0:
16        print(f"Error: Invalid cylinder type '{cylinder_type}'.")
17        return None
18
19    # Calculate the base amount before any deductions or additions.
20    base_amount = price_per_cylinder * num_cylinders
21
22    # 4. Implement the billing formula.
```

```
21
22
23    # 4. Implement the billing formula.
24    # a. Delivery Charges (₹18 to ₹50)
25    delivery_charge = random.uniform(18.0, 50.0)
26
27    # Check if a subsidy is applicable based on the cylinder type.
28    # Subsidy is only applicable for domestic cylinders.
29    # Set subsidy to zero for commercial cylinders.
30    subsidy_amount = 0.0
31
32    # Bill Amount = (Price per Cylinder * Quantity) - Subsidy + Delivery Charges
33    total_bill_amount = (base_amount - subsidy_amount) + delivery_charge
34
35    # Return a dictionary with all the calculated values for easy display.
36    return {
37        "Cylinder Type": cylinder_type,
38        "Number of Cylinders": num_cylinders,
39        "Base Amount": base_amount,
40        "Subsidy": subsidy_amount,
41        "Delivery Charges": delivery_charge
42    }
```

```
43
44
45 def display_bill(bill_details):
46     """
47     Prints an itemized bill in a clean, readable format.
48
49     Args:
50         bill_details (dict): A dictionary containing bill information.
51     """
52     if bill_details is None:
53         return
54
55     print("\n" + "="*30)
56     print("LPG Bill Statement")
57     print("="*30)
58
59     # 5. Calculate and display an itemized bill.
60     print(f"Cylinder Type: {bill_details['Cylinder Type']}")
61     print(f"Number of Cylinders: {bill_details['Number of Cylinders']}")
62     print(f"Base Amount: ₹{bill_details['Base Amount']:,.2f}")
63     print(f"Subsidy: ₹{bill_details['Subsidy']:,.2f}")
64     print(f"Delivery Charges: ₹{bill_details['Delivery Charges']:,.2f}")
65     print(f"Total Bill Amount: ₹{(bill_details['Base Amount'] - bill_details['Subsidy'] + bill_details['Delivery Charges']):,.2f}")
```

Canvas Account Log In x team techniques to impro... x Introducing ChatGPT | Op... x Unimodulpyrn Colab x Google Gemini x Subsidy amount for lpg cy

gemini.google.com/app/5294/594345949.8

Gemini 2.5 Flash

Upgrade

Hello! I can certainly help you with that. I've created a Python program that calculates the LPG bill according to all of your specified instructions. Below, you'll find a report outlining the process and the final, well-commented code.

LPG Bill Calculator Report Aug 11, 10:36 AM Open

LPG Bill Calculator Aug 11, 10:37 AM

I hope this application is exactly what you were looking for! The code is ready to be run as-is, but I've also included comments on how you could modify it to accept user input.

Let me know if you would like to make any changes.

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

LPG Bill Calculator

```
45 def display_bill(bill_details):
46     print(f"Number of cylinders: {bill_details['Number of cylinders']}")
47     print(f"Base Amount: {bill_details['Base Amount']:.2f}")
48     print(f"Subsidy: {bill_details['Subsidy']:.2f}")
49     print(f"Delivery Charges: {bill_details['Delivery Charges']:.2f}")
50     print(f"Total Bill Amount: {bill_details['Total Bill Amount']:.2f}")
51     print(f"---")
52
53 # --- Example Usage ---
54 # You can uncomment the lines below to make the program interactive
55 # by taking user input.
56 # cylinder_type = input("Enter Cylinder Type (e.g., 'Domestic 14.2 kg'): ")
57 # num_cylinders = int(input("Enter Number of Cylinders: "))
58 # subsidy_amount = float(input("Enter Subsidy Amount (0 for commercial): "))
59 # bill_info = calculate_lpg_bill(cylinder_type, num_cylinders, subsidy_amount)
60 # display_bill(bill_info)
61
62 # Example 1: Domestic cylinder with subsidy
63 print(f"--- Example 1: Domestic with subsidy ---")
64 bill_info_1 = calculate_lpg_bill("Domestic 14.2 kg", 1, 25.50)
65
66 Console
```

Canvas Account Log In x team techniques to impro... x Introducing ChatGPT | Op... x Unimodulpyrn Colab x Google Gemini x Subsidy amount for lpg cy

gemini.google.com/app/5294/594345949.8

Gemini 2.5 Flash

Upgrade

Hello! I can certainly help you with that. I've created a Python program that calculates the LPG bill according to all of your specified instructions. Below, you'll find a report outlining the process and the final, well-commented code.

LPG Bill Calculator Report Aug 11, 10:36 AM Open

LPG Bill Calculator Aug 11, 10:37 AM

I hope this application is exactly what you were looking for! The code is ready to be run as-is, but I've also included comments on how you could modify it to accept user input.

Let me know if you would like to make any changes.

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

LPG Bill Calculator

Console

```
--- Example 1: Domestic with subsidy ---
LPG Bill Statement
Cylinder Type: Domestic 14.2 kg
Number of Cylinders: 1
Base Amount: ₹985.00
Subsidy: - ₹25.50
Delivery Charges: + ₹45.41
Total Bill Amount: ₹924.91

--- Example 2: Commercial without subsidy ---
LPG Bill Statement
Cylinder Type: Commercial 19 kg
Number of Cylinders: 2
Base Amount: ₹3773.00
Subsidy: - ₹0.00
Delivery Charges: + ₹18.18
Total Bill Amount: ₹3791.18
```

Canvas Account Log In x team techniques to impro... x Introducing ChatGPT | Op... x Unimodulpyrn Colab x Google Gemini x Subsidy amount for lpg cy

gemini.google.com/app/5294/594345949.8

Gemini 2.5 Flash

Upgrade

Hello! I can certainly help you with that. I've created a Python program that calculates the LPG bill according to all of your specified instructions. Below, you'll find a report outlining the process and the final, well-commented code.

LPG Bill Calculator Report Aug 11, 10:36 AM Open

LPG Bill Calculator Aug 11, 10:37 AM

I hope this application is exactly what you were looking for! The code is ready to be run as-is, but I've also included comments on how you could modify it to accept user input.

Let me know if you would like to make any changes.

Ask Gemini

Deep Research Canvas

Gemini can make mistakes, so double-check it

LPG Bill Calculator

Console

```
--- Example 1: Domestic with subsidy ---
LPG Bill Statement
Cylinder Type: Domestic 14.2 kg
Number of Cylinders: 1
Base Amount: ₹985.00
Subsidy: - ₹25.50
Delivery Charges: + ₹45.41
Total Bill Amount: ₹924.91

--- Example 2: Commercial without subsidy ---
LPG Bill Statement
Cylinder Type: Commercial 19 kg
Number of Cylinders: 2
Base Amount: ₹3773.00
Subsidy: - ₹0.00
Delivery Charges: + ₹18.18
Total Bill Amount: ₹3791.18

--- Code execution complete ---
```

